
Rapport de Travaux Pratiques

PL/SQL

Ce rapport regroupe l'ensemble des travaux pratiques réalisés dans le cadre du cours **Bases de données avancées**.
Chaque TP présente son **objectif**, sa **solution** et sa **conclusion**.

Étudiant : Ilyas YOUSSEFI
Professeur : Pr.Hamid HRIMECH
Module : Bases de Données Avancées
Filière : ISIBD - S6

Année universitaire : 2025–2026

Table des matières

1	TP 1 — Les Vues en Bases de Données	2
	Exercice 1 : Sécurité et Abstraction	3
	Exercice 2 : Analyse Globale et Réflexion Approfondie	3
2	TP 2 — Vues et Index	5
	Exercice 1 : Vues	5
	Exercice 2 : Index	8
3	TP 3 — Variables, RECORD, Collections et Attributs PL/SQL	10
4	TP 4 — PL/SQL Curseurs	12
	Exercice 1 : Curseurs Explicites	12
	Exercice 2 : Curseurs en Boucle FOR	14
5	TP 5 — Curseurs Avancés et Instructions IF-THEN-ELSE	19
	Exercice 1 : Instructions IF-THEN-ELSE	19
	Exercice 2 : Curseurs et Attributs Implicites	21
6	TP 6 — Procédures Stockées et Fonctions en PL/SQL	26
	Exercice 1 : Premier Contact	27
	Exercice 2 : Maximum de Deux Nombres	27
	Exercice 3 : Permutation de Deux Nombres	28
	Exercice 4 : Augmentation de Salaire	29
	Exercice 5 : Conversion en Euros	31
	Exercice 6 : Employés au-dessus d'un Seuil de Salaire	31
	Exercice 7 : Salaire Moyen par Département	32
	Exercice 8 : Mise à Jour d'Adresse par Nom	33
	Exercice 9 : Nombre d'Employés par Département	34
	Exercice 10 : Suppression des Petits Salaires	35
	Synthèse : Procédure vs Fonction	36
7	TP 7 — Les Triggers PL/SQL sous Oracle	38
	Exercices Supplémentaires Proposés	39
	Contrainte 10 : Interdiction de re-s'inscrire à un module déjà validé	40
	Contrainte 11 : Pas de deux examens dans un intervalle de 7 jours	40
	Contrainte 12 : Moyenne calculée uniquement si tous les examens sont passés	41
	Contrainte 13 : Interdiction de supprimer un module utilisé comme prérequis	42

Chapitre 1

TP 1 — Les Vues en Bases de Données

Ce TP a pour objectif de maîtriser les vues en SQL/Oracle, notamment :

- Comprendre et créer des vues simples et complexes
- Utiliser les options `WITH CHECK OPTION` et `WITH READ ONLY`
- Exploiter les vues pour la sécurité et l'abstraction des données
- Analyser les limites des vues et l'intérêt des vues matérialisées

Schéma de la Base de Données

Les exercices s'appuient sur les deux tables suivantes :

```
1 CREATE TABLE Departement (  
2     dept_id    NUMBER PRIMARY KEY,  
3     nom_dept   VARCHAR2(50),  
4     ville      VARCHAR2(50)  
5 );  
6  
7 CREATE TABLE Employe (  
8     emp_id     NUMBER PRIMARY KEY,  
9     nom        VARCHAR2(50),  
10    salaire    NUMBER(8,2),  
11    dept_id     NUMBER,  
12    FOREIGN KEY (dept_id) REFERENCES Departement(dept_id)  
13 );
```

Listing 1.1 – Création des tables Departement et Employe

Exercice 1 : Sécurité et Abstraction

Créer une vue publique sans la colonne salaire pour masquer les informations sensibles.

```
1 CREATE VIEW emp_public AS
2   SELECT emp_id, nom, dept_id
3   FROM   Employe;
```

Listing 1.2 – Vue publique sans le salaire

Discussion — Les vues améliorent-elles les performances ?

Non directement : une vue classique est réévaluée à chaque appel (pas de cache). Elle améliore la **maintenabilité** et la **sécurité**, mais pas les performances brutes. Pour cela, il faut utiliser des **vues matérialisées**.

Exercice 2 : Analyse Globale et Réflexion Approfondie

1. Architecture de vues pour la sécurité :

- `vue_manager` : filtre par `dept_id` du manager connecté.
- `vue_rh` : accès complet à toutes les colonnes et tous les départements.
- `vue_employe` : exclut la colonne `salaire` des autres employés.

2. Limites des vues classiques sur grand volume :

- La requête sous-jacente est ré-exécutée à chaque accès.
- Pas de stockage physique des résultats → latence élevée sur des millions de lignes.
- Les agrégations coûteuses (AVG, GROUP BY) sont recalculées en permanence.

3. Intérêt des vues matérialisées :

Une vue matérialisée stocke physiquement le résultat de la requête. Elle est rafraîchie périodiquement (ON COMMIT ou à la demande). Elle réduit drastiquement le temps de réponse pour des requêtes analytiques lourdes.

4. Vues vs GRANT/REVOKE :

- GRANT/REVOKE contrôle l'accès à une table entière (toutes les colonnes).
- Une vue permet un contrôle plus fin : filtrage par ligne **et** par colonne.
- Les deux mécanismes sont complémentaires et doivent être combinés.

5. Une vue peut-elle garantir la sécurité complète ?

Non. Une vue est une couche d'abstraction, mais elle ne remplace pas : les privilèges Oracle (GRANT), l'audit des accès, le chiffrement des données sensibles, et les politiques de sécurité réseau.

6. Stratégie d'optimisation :

- Créer des **index** sur les colonnes de jointure et de filtrage.

- Utiliser le **partitionnement** par site ou département.
- Remplacer les vues analytiques fréquentes par des **vues matérialisées**.
- Activer le **query rewriting** Oracle pour rediriger automatiquement les requêtes vers les vues matérialisées.

Ce TP a mis en évidence le rôle central des vues dans une base de données Oracle :

- Les vues simples et complexes permettent d'abstraire et de simplifier l'accès aux données pour les utilisateurs finaux.
- Les options **WITH CHECK OPTION** et **WITH READ ONLY** renforcent l'intégrité et la sécurité des données accessibles via les vues.
- Sur de grands volumes, les vues classiques montrent leurs limites en termes de performance ; les vues matérialisées constituent alors une alternative efficace.
- La sécurité complète d'un système repose sur une combinaison de vues, de privilèges **GRANT/REVOKE**, d'audit et de chiffrement.

Chapitre 2

TP 2 — Vues et Index

Ce TP a pour objectif de consolider l'utilisation des vues et d'introduire les index en Oracle, notamment :

- Créer des vues simples et calculées sur un schéma multi-tables
- Tester les mises à jour et insertions à travers les vues
- Maîtriser les options `WITH CHECK OPTION` et leurs effets
- Créer, utiliser et supprimer des index avec ou sans unicité

Schéma Relationnel

```
1 EMP(Mat, NomE, Poste, DatEmb, Sup, Salaire, Commission,
   NumDept)
2 DEPT(NumDept, NomDept, Lieu)
3 PROJET(CodeP, NomP)
4 PARTICIPATION(Mat, CodeP, Fonction)
```

Listing 2.1 – Schéma de la base de données

Exercice 1 : Vues

Question 1 : Vue V_EMP

Créer une vue `V_EMP` contenant le matricule, le nom, le numéro de département, la somme salaire+commission baptisée `GAINS`, et le lieu du département.

```
1 CREATE VIEW V_EMP AS
2   SELECT e.Mat,
3          e.NomE,
```

```
4      e.NumDept ,
5      (e.Salaire + NVL(e.Commission, 0)) AS GAINS ,
6      d.Lieu
7  FROM    EMP e
8  JOIN    DEPT d ON e.NumDept = d.NumDept;
```

Listing 2.2 – Création de la vue V_EMP

Question 2 : Sélection sur V_EMP

Sélectionner les lignes de V_EMP dont le salaire total est supérieur à 10 000 F.

```
1  SELECT *
2  FROM    V_EMP
3  WHERE    GAINS > 10000;
```

Listing 2.3 – Interrogation de V_EMP

Question 3 : Mise à jour via V_EMP

Tenter de mettre à jour le nom de l'employé MARTIN à travers la vue.

```
1  UPDATE V_EMP
2  SET     NomE = 'MARTIN_NEW'
3  WHERE   NomE = 'MARTIN';
```

Listing 2.4 – Mise à jour via la vue V_EMP

Résultat : La mise à jour **réussit** car V_EMP est basée sur une jointure simple et la colonne NomE appartient à une seule table de base (EMP). Oracle peut propager la modification directement.

Question 4 : Vue VEMP10 sans CHECK OPTION

Créer une vue ne contenant que les employés du département 10, sans CHECK OPTION, puis insérer un employé du département 20.

```
1  CREATE VIEW VEMP10 AS
2  SELECT * FROM EMP
3  WHERE   NumDept = 10;
4
5  INSERT INTO VEMP10 (Matr, NomE, Poste, Salaire, NumDept)
6  VALUES (999, 'SOULIER', 'EMPLOYE', 3000, 20);
```

Listing 2.5 – Vue VEMP10 sans CHECK OPTION

Résultat :

- `SELECT * FROM VEMP10 → SOULIER n'apparaît pas` (il est dans le dept 20, filtré par la vue).
- `SELECT * FROM EMP WHERE NomE = 'SOULIER' → SOULIER est bien présent` dans la table de base.

Sans `CHECK OPTION`, Oracle accepte l'insertion même si la ligne ne satisfait pas le filtre de la vue.

Question 5 : Recréation de VEMP10 avec CHECK OPTION

```
1 DROP VIEW VEMP10;  
2  
3 CREATE VIEW VEMP10 AS  
4     SELECT * FROM EMP  
5     WHERE NumDept = 10  
6     WITH CHECK OPTION;
```

Listing 2.6 – Suppression et recréation avec WITH CHECK OPTION

Question 6 : Tentative d'insertion et de modification

```
1 INSERT INTO VEMP10 (Matr, NomE, Poste, Salaire, NumDept)  
2 VALUES (888, 'BALARD', 'EMPLOYE', 2800, 30);
```

Listing 2.7 – Insertion hors condition avec CHECK OPTION

Résultat : Oracle lève l'erreur `ORA-01402 : view WITH CHECK OPTION where-clause violation`. L'insertion est rejetée car `NumDept = 30` viole la condition `WHERE NumDept = 10`.

```
1 UPDATE VEMP10  
2 SET     NumDept = 20  
3 WHERE  NomE = 'DUPONT';
```

Listing 2.8 – Tentative de modification du département

Résultat : Même erreur `ORA-01402`. Modifier le département ferait sortir la ligne du périmètre de la vue, ce que `WITH CHECK OPTION` interdit.

Question 7 : Pourcentage du salaire par rapport au total du département

Solution 1 — avec une vue intermédiaire :

```
1 CREATE VIEW TOTAL_DEPT AS
2   SELECT NumDept, SUM(Salaire) AS Total_Salaire
3   FROM   EMP
4   GROUP BY NumDept;
5
6 SELECT e.NomE,
7        e.Salaire,
8        e.NumDept,
9        ROUND(e.Salaire / t.Total_Salaire * 100, 2) AS
           Pourcentage
10 FROM   EMP e
11 JOIN   TOTAL_DEPT t ON e.NumDept = t.NumDept
12 ORDER BY e.NumDept, Pourcentage DESC;
```

Listing 2.9 – Solution avec vue intermédiaire

Solution 2 — avec sous-requête dans le FROM :

```
1 SELECT e.NomE,
2        e.Salaire,
3        e.NumDept,
4        ROUND(e.Salaire / t.Total_Salaire * 100, 2) AS
           Pourcentage
5 FROM   EMP e
6 JOIN   (SELECT NumDept, SUM(Salaire) AS Total_Salaire
7         FROM   EMP
8         GROUP BY NumDept) t
9       ON e.NumDept = t.NumDept
10 ORDER BY e.NumDept, Pourcentage DESC;
```

Listing 2.10 – Solution sans vue, sous-requête dans le FROM

Exercice 2 : Index

Question 1 : Création des Index

```
1 CREATE UNIQUE INDEX idx_emp_nom
2   ON EMP (NomE);
3
4 CREATE INDEX idx_emp_dept
5   ON EMP (NumDept);
```

Listing 2.11 – Création d'un index unique sur NomE et d'un index sur NumDept

Remarque : L'index unique sur NomE interdit deux employés portant exactement le même nom. À utiliser avec précaution dans un contexte réel.

Question 2 : Suppression de l'Index sur NomE

```
1 DROP INDEX idx_emp_nom_e;
```

Listing 2.12 – Suppression de l'index sur NomE

Effet : La contrainte d'unicité sur NomE disparaît. Les requêtes sur ce champ utiliseront désormais un *full table scan* sauf si un autre index existe. La table EMP reste intacte.

Ce TP a permis de consolider deux mécanismes fondamentaux d'Oracle :

- Les vues offrent une abstraction puissante : elles peuvent être interrogées, et parfois modifiées, comme des tables ordinaires. L'option `WITH CHECK OPTION` est indispensable pour garantir la cohérence des données modifiées via une vue filtrée.
- Sans `WITH CHECK OPTION`, une insertion hors filtre est silencieusement acceptée et la ligne devient invisible dans la vue tout en existant dans la table de base — comportement à éviter.
- Les index accélèrent les recherches et peuvent imposer l'unicité, mais leur création doit être réfléchie : un index inutile consomme de l'espace et ralentit les opérations DML.
- La combinaison vues + index + privilèges constitue le triptyque de base pour une base de données performante et sécurisée.

Chapitre 3

TP 3 — Variables, RECORD, Collections et Attributs PL/SQL

Ce TP a pour objectif de maîtriser les concepts fondamentaux de la déclaration et de la manipulation des données en PL/SQL, notamment :

- La déclaration et la portée des variables scalaires
- L'utilisation des attributs %TYPE et %ROWTYPE
- La création et l'utilisation des enregistrements (RECORD)
- Les tables PL/SQL (collections associatives)
- Les bonnes pratiques et les erreurs fréquentes à éviter

Exercice Final de Synthèse

Travail demandé :

1. Créer un RECORD personnalisé pour un employé.
2. Créer une TABLE de ce RECORD.
3. Insérer 5 employés avec des données variées.
4. Calculer le salaire moyen.
5. Identifier le salaire maximum.

```
1 DECLARE
2     TYPE t_employe IS RECORD (
3         id          NUMBER,
4         nom          VARCHAR2(50),
5         salaire      NUMBER(8,2)
6     );
```

```
7  TYPE t_table_emp IS TABLE OF t_employe
8      INDEX BY BINARY_INTEGER;
9
10  v_equipe    t_table_emp;
11  v_total     NUMBER(10,2) := 0;
12  v_max_sal   NUMBER(8,2)  := 0;
13  v_moyenne   NUMBER(8,2);
14  BEGIN
15      v_equipe(1).id := 1; v_equipe(1).nom := 'Alice';
16          v_equipe(1).salaire := 3500;
17      v_equipe(2).id := 2; v_equipe(2).nom := 'Bob';
18          v_equipe(2).salaire := 4200;
19      v_equipe(3).id := 3; v_equipe(3).nom := 'Carla';
20          v_equipe(3).salaire := 5100;
21      v_equipe(4).id := 4; v_equipe(4).nom := 'David';
22          v_equipe(4).salaire := 3800;
23      v_equipe(5).id := 5; v_equipe(5).nom := 'Eva';
24          v_equipe(5).salaire := 6000;
25
26  FOR i IN 1..5 LOOP
27      v_total := v_total + v_equipe(i).salaire;
28      IF v_equipe(i).salaire > v_max_sal THEN
29          v_max_sal := v_equipe(i).salaire;
30      END IF;
31  END LOOP;
32
33  v_moyenne := v_total / 5;
34
35  DBMS_OUTPUT.PUT_LINE('Salaire moyen : ' || v_moyenne);
36  DBMS_OUTPUT.PUT_LINE('Salaire maximum: ' || v_max_sal);
37  END;
```

Listing 3.1 – Solution de l'exercice final de synthèse

Ce TP a permis de poser les bases essentielles de la programmation PL/SQL :

- La déclaration rigoureuse des variables avec les bons types et préfixes (`v_`, `c_`, `t_`) garantit un code lisible et maintenable.
- L'utilisation de `%TYPE` et `%ROWTYPE` assure l'adaptabilité automatique aux évolutions du schéma de base de données.
- Les structures `RECORD` permettent de regrouper logiquement des données hétérogènes, tandis que les tables associatives offrent une collection dynamique et flexible.

Chapitre 4

TP 4 — PL/SQL Curseurs

Ce TP a pour objectif de maîtriser les curseurs en PL/SQL, notamment :

- Comprendre le cycle de vie d'un curseur : déclaration, ouverture, extraction (FETCH) et fermeture
- Utiliser les curseurs explicites avec gestion manuelle
- Utiliser les curseurs dans une boucle FOR (gestion semi-automatique)
- Appliquer les curseurs sur des requêtes filtrées, agrégées et imbriquées

Schémas des Bases de Données

Base Usine (Exercice 1) :

```
1 Commande(Numcmd, Datcmd, Numfour, Mncmd)
2 Fournisseurs(Numfour, Adrfour)
```

Listing 4.1 – Schéma base Usine

Base Gavasoft (Exercice 2) :

```
1 Emp(NumE, NomE, Fonction, NumS, Embauche, Salaire, Comm, NumD)
2 Dept(NumD, NomD, Lieu)
```

Listing 4.2 – Schéma base Gavasoft

Exercice 1 : Curseurs Explicites — Base Usine

Question 1 : Liste de toutes les commandes

```
1 DECLARE
2     CURSOR c_commandes IS
3         SELECT Numcmd, Datcmd, Numfour, Mncmd
4         FROM   Commande;
5     v_cmd Commande%ROWTYPE;
6 BEGIN
7     OPEN c_commandes;
8     LOOP
9         FETCH c_commandes INTO v_cmd;
10        EXIT WHEN c_commandes%NOTFOUND;
11        DBMS_OUTPUT.PUT_LINE(
12            'Cmd: ' || v_cmd.Numcmd ||
13            ' | Date: ' || v_cmd.Datcmd ||
14            ' | Fournisseur: ' || v_cmd.Numfour ||
15            ' | Montant: ' || v_cmd.Mncmd
16        );
17    END LOOP;
18    CLOSE c_commandes;
19 END;
20 /
```

Listing 4.3 – Curseur explicite : liste des commandes

Question 2 : Commandes avec montant supérieur à la moyenne

```
1 DECLARE
2     v_moyenne NUMBER;
3     CURSOR c_sup_moy IS
4         SELECT Numcmd, Datcmd, Mncmd
5         FROM   Commande
6         WHERE  Mncmd > v_moyenne;
7     v_cmd c_sup_moy%ROWTYPE;
8 BEGIN
9     SELECT AVG(Mncmd) INTO v_moyenne FROM Commande;
10
11    OPEN c_sup_moy;
12    LOOP
13        FETCH c_sup_moy INTO v_cmd;
14        EXIT WHEN c_sup_moy%NOTFOUND;
15        DBMS_OUTPUT.PUT_LINE(
16            'Cmd: ' || v_cmd.Numcmd ||
17            ' | Montant: ' || v_cmd.Mncmd
18        );
19    END LOOP;
```

```
20 CLOSE c_sup_moy;  
21 END;  
22 /
```

Listing 4.4 – Curseur : commandes au-dessus de la moyenne

Question 3 : Commandes avec fournisseur parisien

```
1 DECLARE  
2 CURSOR c_paris IS  
3     SELECT c.Numcmd, c.Datcmd, c.Mncmd, f.Adrfour  
4     FROM   Commande c  
5     JOIN   Fournisseurs f ON c.Numfour = f.Numfour  
6     WHERE  f.Adrfour = 'Paris';  
7     v_row c_paris%ROWTYPE;  
8 BEGIN  
9     OPEN c_paris;  
10    LOOP  
11        FETCH c_paris INTO v_row;  
12        EXIT WHEN c_paris%NOTFOUND;  
13        DBMS_OUTPUT.PUT_LINE(  
14            'Cmd: ' || v_row.Numcmd ||  
15            ' | Montant: ' || v_row.Mncmd ||  
16            ' | Ville: ' || v_row.Adrfour  
17        );  
18    END LOOP;  
19    CLOSE c_paris;  
20 END;  
21 /
```

Listing 4.5 – Curseur : commandes d'un fournisseur parisien

Exercice 2 : Curseurs en Boucle FOR — Base Gavasoft

Question 1 : Employés avec commission, triés par commission décroissante

```
1 BEGIN  
2     FOR rec IN (  
3         SELECT NomE, Comm  
4         FROM   Emp  
5         WHERE  Comm IS NOT NULL  
6         ORDER BY Comm DESC
```

```
7 ) LOOP
8   DBMS_OUTPUT.PUT_LINE(
9     rec.NomE || ' - Commission : ' || rec.Comm
10  );
11 END LOOP;
12 END;
13 /
```

Listing 4.6 – FOR curseur : employés avec commission décroissante

Question 2 : Personnes embauchées depuis le 01/09/2006

```
1 BEGIN
2   FOR rec IN (
3     SELECT NomE, Embauche
4     FROM   Emp
5     WHERE  Embauche >= TO_DATE('01/09/2006', 'DD/MM/YYYY')
6   ) LOOP
7     DBMS_OUTPUT.PUT_LINE(
8       rec.NomE || ' - Embauch le : ' || rec.Embauche
9     );
10  END LOOP;
11 END;
12 /
```

Listing 4.7 – FOR curseur : embauches depuis le 01-09-2006

Question 3 : Employés travaillant à Créteil

```
1 BEGIN
2   FOR rec IN (
3     SELECT e.NomE, d.Lieu
4     FROM   Emp e
5     JOIN   Dept d ON e.NumD = d.NumD
6     WHERE  d.Lieu = 'Cr teil'
7   ) LOOP
8     DBMS_OUTPUT.PUT_LINE(
9       rec.NomE || ' travaille ' || rec.Lieu
10    );
11  END LOOP;
12 END;
13 /
```

Listing 4.8 – FOR curseur : employés à Créteil

Question 4 : Subordonnés de “Guimezanes”

```
1 DECLARE
2     v_numE Emp.NumE%TYPE;
3 BEGIN
4     SELECT NumE INTO v_numE
5     FROM Emp
6     WHERE NomE = 'Guimezanes';
7
8     FOR rec IN (
9         SELECT NomE, Fonction
10        FROM Emp
11        WHERE NumS = v_numE
12    ) LOOP
13        DBMS_OUTPUT.PUT_LINE(
14            rec.NomE || ' - ' || rec.Fonction
15        );
16    END LOOP;
17 END;
18 /
```

Listing 4.9 – FOR curseur : subordonnés de Guimezanes

Question 5 : Moyenne des salaires

```
1 DECLARE
2     v_total  NUMBER := 0;
3     v_count  NUMBER := 0;
4 BEGIN
5     FOR rec IN (SELECT Salaire FROM Emp) LOOP
6         v_total := v_total + NVL(rec.Salaire, 0);
7         v_count := v_count + 1;
8     END LOOP;
9
10    IF v_count > 0 THEN
11        DBMS_OUTPUT.PUT_LINE(
12            'Moyenne des salaires : ' || ROUND(v_total /
13                v_count, 2)
14        );
15    END IF;
16 END;
17 /
```

Listing 4.10 – FOR curseur : calcul de la moyenne des salaires

Question 6 : Nombre de commissions non NULL

```
1 DECLARE
2   v_count NUMBER := 0;
3 BEGIN
4   FOR rec IN (
5     SELECT Comm FROM Emp WHERE Comm IS NOT NULL
6   ) LOOP
7     v_count := v_count + 1;
8   END LOOP;
9
10  DBMS_OUTPUT.PUT_LINE(
11    'Nombre de commissions non NULL : ' || v_count
12  );
13 END;
14 /
```

Listing 4.11 – FOR curseur : comptage des commissions non NULL

Question 7 : Employés gagnant plus que la moyenne

```
1 DECLARE
2   v_moyenne NUMBER;
3 BEGIN
4   SELECT AVG(Salaire) INTO v_moyenne FROM Emp;
5
6   DBMS_OUTPUT.PUT_LINE(
7     'Moyenne entreprise : ' || ROUND(v_moyenne, 2)
8   );
9
10  FOR rec IN (
11    SELECT NomE, Salaire
12    FROM   Emp
13    WHERE  Salaire > v_moyenne
14    ORDER BY Salaire DESC
15  ) LOOP
16    DBMS_OUTPUT.PUT_LINE(
17      rec.NomE || ' - Salaire : ' || rec.Salaire
18    );
19  END LOOP;
20 END;
21 /
```

Listing 4.12 – FOR curseur : employés au-dessus de la moyenne salariale

Ce TP a permis de maîtriser les curseurs PL/SQL sous leurs deux formes principales :

- Le curseur explicite (**OPEN** / **FETCH** / **CLOSE**) offre un contrôle total sur le parcours des lignes et est indispensable pour les traitements complexes nécessitant une logique conditionnelle fine.
- La boucle **FOR curseur** simplifie considérablement le code en gérant automatiquement l'ouverture, le fetch et la fermeture ; elle est privilégiée pour les traitements séquentiels simples.
- L'attribut **%NOTFOUND** est essentiel pour terminer proprement une boucle de fetch manuel et éviter les lectures indéfinies.
- Les curseurs paramétrés et les sous-requêtes dans la déclaration du curseur permettent de construire des requêtes dynamiques et réutilisables.

Chapitre 5

TP 5 — Curseurs Avancés et Instructions IF-THEN-ELSE

Ce TP a pour objectif d'approfondir l'utilisation des structures de contrôle et des curseurs en PL/SQL, notamment :

- Utiliser l'instruction IF-THEN-ELSE pour des traitements conditionnels
- Manipuler les curseurs implicites et leurs attributs (SQL%ROWCOUNT, SQL%FOUND, SQL%NOTFOUND)
- Utiliser la clause WHERE CURRENT OF pour les mises à jour positionnées
- Appliquer les curseurs sur des cas pratiques variés

Exercice 1 : Instructions IF-THEN-ELSE

Question 1 : Réorganisation de deux variables

Réorganiser deux variables de sorte que `small_nbr` contienne le plus petit et `big_nbr` le plus grand.

```
1 DECLARE
2     small_nbr NUMBER := 25;
3     big_nbr   NUMBER := 5;
4     v_temp    NUMBER;
5 BEGIN
6     IF small_nbr > big_nbr THEN
7         v_temp := small_nbr;
8         small_nbr := big_nbr;
9         big_nbr := v_temp;
10    END IF;
```

```
11
12     DBMS_OUTPUT.PUT_LINE('small_nbr = ' || small_nbr);
13     DBMS_OUTPUT.PUT_LINE('big_nbr    = ' || big_nbr);
14 END;
15 /
```

Listing 5.1 – IF-THEN-ELSE : permutation de deux variables

Résultat attendu :

small_nbr = 5

big_nbr = 25

Question 2 : Nombre pair ou impair

```
1 DECLARE
2     nbr NUMBER := 3;
3 BEGIN
4     IF MOD(nbr, 2) = 0 THEN
5         DBMS_OUTPUT.PUT_LINE('Le nombre ' || nbr || ' est un
6             nombre pair. ');
7     ELSE
8         DBMS_OUTPUT.PUT_LINE('Le nombre ' || nbr || ' est un
9             nombre impair. ');
10    END IF;
11 END;
12 /
```

Listing 5.2 – IF-THEN-ELSE : parité d'un nombre

Question 3 : Détection du week-end

```
1 DECLARE
2     date1    DATE      := TO_DATE('2-Mar-2024', 'DD-Mon-YYYY');
3     v_jour    VARCHAR2(20);
4 BEGIN
5     v_jour := TO_CHAR(date1, 'DAY',
6         'NLS_DATE_LANGUAGE=ENGLISH');
7
8     IF TRIM(v_jour) IN ('SATURDAY', 'SUNDAY') THEN
9         DBMS_OUTPUT.PUT_LINE(
10             'Le jour de la date donne est ' || TRIM(v_jour) ||
11             ' : tombe le week-end.'
12         );
13     ELSE
14         DBMS_OUTPUT.PUT_LINE(
```

```
14      'Le jour de la date donne est ' || TRIM(v_jour) ||  
15      ' : ne tombe pas le week-end.'  
16  );  
17  END IF;  
18  END;  
19  /
```

Listing 5.3 – IF-THEN-ELSE : détection samedi ou dimanche

Question 4 : Copie de la table clients vers tmp_clients

```
1  -- Cr ation de la table destination  
2  CREATE TABLE tmp_clients AS SELECT * FROM clients WHERE 1  
   = 0;  
3  
4  DECLARE  
5      CURSOR c_clients IS  
6          SELECT client_id, nom, ville, age FROM clients;  
7      v_row c_clients%ROWTYPE;  
8  BEGIN  
9      OPEN c_clients;  
10     LOOP  
11         FETCH c_clients INTO v_row;  
12         EXIT WHEN c_clients%NOTFOUND;  
13  
14         INSERT INTO tmp_clients (client_id, nom, ville, age)  
15         VALUES (v_row.client_id, v_row.nom, v_row.ville,  
16                 v_row.age);  
17     END LOOP;  
18     CLOSE c_clients;  
19  
20     COMMIT;  
21     DBMS_OUTPUT.PUT_LINE(  
22         c_clients%ROWCOUNT || ' lignes copi es.'  
23     );  
24  END;  
25  /
```

Listing 5.4 – Curseur : copie de clients vers tmp_clients

Exercice 2 : Curseurs et Attributs Implicites

Question 1 : Afficher les noms de tous les pays

```
1 BEGIN
2   FOR rec IN (SELECT pays_id, nom FROM pays) LOOP
3     DBMS_OUTPUT.PUT_LINE(rec.pays_id || ' - ' || rec.nom);
4   END LOOP;
5 END;
6 /
```

Listing 5.5 – Curseur FOR : liste des pays

Question 2 : Identifiants et noms de pays avec titre

```
1 BEGIN
2   DBMS_OUTPUT.PUT_LINE('ID Pays | Nom du Pays');
3   DBMS_OUTPUT.PUT_LINE('-----|-----');
4   FOR rec IN (
5     SELECT pays_id, nom FROM pays ORDER BY nom
6   ) LOOP
7     DBMS_OUTPUT.PUT_LINE(
8       RPAD(rec.pays_id, 9) || ' | ' || rec.nom
9     );
10  END LOOP;
11 END;
12 /
```

Listing 5.6 – Curseur FOR : pays avec en-tête

Question 3 : Nombre de lignes affectées avec SQL%ROWCOUNT

```
1 BEGIN
2   UPDATE pays SET nom = UPPER(nom)
3   WHERE pays_id > 1003;
4
5   DBMS_OUTPUT.PUT_LINE(
6     'Nombre de lignes modifi es : ' || SQL%ROWCOUNT
7   );
8   ROLLBACK;
9 END;
10 /
```

Listing 5.7 – Curseur implicite : SQL%ROWCOUNT après UPDATE

Remarque : SQL%ROWCOUNT est un attribut du curseur implicite géré automatiquement par Oracle après chaque instruction DML.

Question 4 : Employés avec leur département

```
1 BEGIN
2   DBMS_OUTPUT.PUT_LINE(
3     RPAD('ID', 6) || RPAD('Nom', 10) ||
4     RPAD('Pr nom', 10) || 'D partement'
5   );
6   DBMS_OUTPUT.PUT_LINE(RPAD('-', 40, '-'));
7
8   FOR rec IN (
9     SELECT e.id, e.nom, e.prenom, d.nom AS dept
10    FROM   employees e
11    JOIN   departements d ON e.departement_id = d.dep_id
12    ORDER BY e.id
13  ) LOOP
14    DBMS_OUTPUT.PUT_LINE(
15      RPAD(rec.id, 6) || RPAD(rec.nom, 10) ||
16      RPAD(rec.prenom, 10) || rec.dept
17    );
18  END LOOP;
19 END;
20 /
```

Listing 5.8 – Curseur FOR : employés avec nom de département

Question 5 : SQL%FOUND après DELETE

```
1 BEGIN
2   DELETE FROM pays WHERE pays_id = 1005;
3
4   IF SQL%FOUND THEN
5     DBMS_OUTPUT.PUT_LINE(
6       'Suppression r ussie : ' || SQL%ROWCOUNT || '
7       ligne(s) supprim e(s).'
8     );
9   ELSE
10    DBMS_OUTPUT.PUT_LINE('Aucune ligne supprim e.');
```

Listing 5.9 – Curseur implicite : SQL%FOUND après DELETE

Question 6 : SQL%NOTFOUND après UPDATE

```
1 BEGIN
2   UPDATE pays SET nom = 'Inconnu'
3   WHERE pays_id = 9999;
4
5   IF SQL%NOTFOUND THEN
6     DBMS_OUTPUT.PUT_LINE(
7       'Aucune ligne mise à jour : pays introuvable.'
8     );
9   ELSE
10    DBMS_OUTPUT.PUT_LINE(
11      'Mise à jour réussie : ' || SQL%ROWCOUNT || '
12      ligne(s).'
13    );
14  END IF;
15
16  ROLLBACK;
17 END;
```

Listing 5.10 – Curseur implicite : SQL%NOTFOUND après UPDATE

Question 7 : Augmentation de salaire avec WHERE CURRENT OF

Augmenter le salaire des employés du département 8 : +100\$ si salaire < 5000\$, sinon +50\$, en utilisant WHERE CURRENT OF.

```
1 DECLARE
2   CURSOR c_emp IS
3     SELECT id, salaire
4     FROM   employees
5     WHERE  departement_id = 8
6     FOR UPDATE OF salaire;
7
8   v_emp c_emp%ROWTYPE;
9 BEGIN
10  OPEN c_emp;
11  LOOP
12    FETCH c_emp INTO v_emp;
13    EXIT WHEN c_emp%NOTFOUND;
14
15    IF v_emp.salaire < 5000 THEN
16      UPDATE employees
17      SET      salaire = salaire + 100
```

```
18      WHERE CURRENT OF c_emp;  
19      DBMS_OUTPUT.PUT_LINE(  
20          'Employ ' || v_emp.id || ' : +100$ (salaire <  
21              5000)'  
22      );  
23  ELSE  
24      UPDATE employees  
25      SET      salaire = salaire + 50  
26      WHERE CURRENT OF c_emp;  
27      DBMS_OUTPUT.PUT_LINE(  
28          'Employ ' || v_emp.id || ' : +50$ (salaire >=  
29              5000)'  
30      );  
31  END IF;  
32  END LOOP;  
33  CLOSE c_emp;  
34  COMMIT;  
35  END;  
36  /
```

Listing 5.11 – WHERE CURRENT OF : mise à jour positionnée

Remarque : La clause `FOR UPDATE` verrouille les lignes sélectionnées. `WHERE CURRENT OF` cible précisément la ligne courante du curseur sans réécrire la condition `WHERE`, ce qui est plus sûr et plus performant.

Ce TP a permis d'élargir la maîtrise des curseurs PL/SQL à des cas pratiques avancés :

- L'instruction `IF-THEN-ELSE` est la base de tout traitement conditionnel ; combinée aux curseurs, elle permet des traitements ligne par ligne avec logique métier.
- Les attributs du curseur implicite (`SQL%ROWCOUNT`, `SQL%FOUND`, `SQL%NOTFOUND`) fournissent un retour immédiat sur l'effet des instructions DML sans nécessiter de curseur explicite.
- La clause `WHERE CURRENT OF` associée à `FOR UPDATE` est la méthode la plus fiable pour mettre à jour ou supprimer la ligne courante d'un curseur, évitant les risques de concurrence.
- La copie de tables via curseur illustre un cas d'usage réel de migration ou d'archivage de données en PL/SQL.

Chapitre 6

TP 6 — Procédures Stockées et Fonctions en PL/SQL

Ce TP a pour objectif de maîtriser les sous-programmes PL/SQL, notamment :

- La syntaxe des blocs anonymes, procédures et fonctions
- Les modes de paramètres : IN, OUT, IN OUT
- La manipulation de données (SELECT, UPDATE, DELETE) dans des sous-programmes
- La gestion des exceptions et la distinction procédure vs fonction

Structure de la Table et Données Initiales

```
1 CREATE TABLE EMPLOYES (  
2     ENUM      VARCHAR2(5) PRIMARY KEY,  
3     ENAME     VARCHAR2(30),  
4     SALARY    NUMBER(8,2),  
5     ADDRESS   VARCHAR2(30),  
6     DEPT     VARCHAR2(5)  
7 );  
8  
9 INSERT INTO EMPLOYES VALUES ('E7', 'Amine', 7500, 'Fes', 'D2');  
10 INSERT INTO EMPLOYES VALUES ('E6', 'Aziz', 8500, 'Casa', 'D1');  
11 INSERT INTO EMPLOYES VALUES ('E5', 'Amina', 8000, 'Rabat', 'D3');  
12 INSERT INTO EMPLOYES VALUES ('E4', 'Said', 5500, 'Agadir', 'D3');  
13 INSERT INTO EMPLOYES VALUES ('E3', 'Fatima', 7000, 'Tanger', 'D2');  
14 INSERT INTO EMPLOYES VALUES ('E2', 'Ahmed', 6000, 'Casa', 'D1');  
15 INSERT INTO EMPLOYES VALUES ('E1', 'Ali', 8000, 'Rabat', 'D1');  
16 INSERT INTO EMPLOYES VALUES ('E8', 'Ahmed', 4400, 'Casa', 'D4');  
17 COMMIT;
```

Listing 6.1 – Création et alimentation de la table EMPLOYES

Exercice 1 : Premier Contact

1. Bloc anonyme affichant Hello :

```
1 BEGIN
2   DBMS_OUTPUT.PUT_LINE('Hello');
3 END;
4 /
```

Listing 6.2 – Bloc anonyme : affichage Hello

2. Fonction retournant le carré d'un nombre :

```
1 CREATE OR REPLACE FUNCTION fn_carre(p_nbr IN NUMBER)
2 RETURN NUMBER IS
3 BEGIN
4   RETURN p_nbr * p_nbr;
5 END;
6 /
7
8 SELECT fn_carre(7) AS carre FROM DUAL;
```

Listing 6.3 – Fonction : carré d'un nombre

Questions supplémentaires :

- Si on omet RETURN dans une fonction, Oracle lève une erreur à la compilation : PLS-00503: RETURN statement required.
- Une procédure peut renvoyer une valeur via OUT, mais elle ne peut pas être appelée dans une expression SQL. La fonction, elle, s'intègre directement dans un SELECT.
- FROM DUAL est une table fictive d'une seule ligne/colonne fournie par Oracle, utilisée pour évaluer des expressions ou appeler des fonctions sans table réelle.

Exercice 2 : Maximum de Deux Nombres

Procédure avec paramètre OUT :

```
1 CREATE OR REPLACE PROCEDURE pr_maximum(
2   p_a   IN   NUMBER,
3   p_b   IN   NUMBER,
4   p_max OUT NUMBER
5 ) IS
```

```

6 BEGIN
7     IF p_a >= p_b THEN
8         p_max := p_a;
9     ELSE
10        p_max := p_b;
11    END IF;
12 END;
13 /
14
15 DECLARE
16     v_max NUMBER;
17 BEGIN
18     pr_maximum(12, 27, v_max);
19     DBMS_OUTPUT.PUT_LINE('Maximum : ' || v_max);
20 END;
21 /

```

Listing 6.4 – Procédure : maximum de deux nombres

Version fonction :

```

1 CREATE OR REPLACE FUNCTION fn_maximum(p_a IN NUMBER, p_b
      IN NUMBER)
2 RETURN NUMBER IS
3 BEGIN
4     RETURN GREATEST(p_a, p_b);
5 END;
6 /
7
8 SELECT fn_maximum(12, 27) FROM DUAL;

```

Listing 6.5 – Fonction : maximum de deux nombres

Questions supplémentaires :

- Si le paramètre OUT n'est pas initialisé dans la procédure, sa valeur reste NULL à la sortie.
- La portée d'un paramètre OUT est limitée au bloc appelant ; il n'est pas accessible en dehors de la session.

Exercice 3 : Permutation de Deux Nombres

```

1 CREATE OR REPLACE PROCEDURE pr_permuter(
2     p_x IN OUT NUMBER,
3     p_y IN OUT NUMBER
4 ) IS
5     v_temp NUMBER;
6 BEGIN

```

```

7   v_temp := p_x;
8   p_x    := p_y;
9   p_y    := v_temp;
10  END;
11  /
12
13  DECLARE
14     v_a NUMBER := 10;
15     v_b NUMBER := 99;
16  BEGIN
17     DBMS_OUTPUT.PUT_LINE('Avant : a=' || v_a || ', b=' ||
18                           v_b);
19     pr_permuter(v_a, v_b);
20     DBMS_OUTPUT.PUT_LINE('Apr s : a=' || v_a || ', b=' ||
21                           v_b);
22  END;
23  /

```

Listing 6.6 – Procédure : permutation avec IN OUT

Questions supplémentaires :

- Avec IN seul, les paramètres sont en lecture seule : toute affectation provoquerait une erreur de compilation.
- Un autre exemple d'IN OUT : normaliser une chaîne de caractères (trim + upper) passée en paramètre.
- On peut permuter deux colonnes d'une table via une procédure en utilisant une variable temporaire et deux UPDATE successifs dans une transaction.

Exercice 4 : Augmentation de Salaire

```

1  CREATE OR REPLACE PROCEDURE augmenter_salaire(
2     p_nom   IN VARCHAR2,
3     p_taux  IN NUMBER
4  ) IS
5  BEGIN
6     UPDATE EMPLOYEES
7     SET    SALARY = SALARY * (1 + p_taux / 100)
8     WHERE  ENAME = p_nom
9     AND    ROWNUM = 1;
10
11     IF SQL%ROWCOUNT = 0 THEN
12         DBMS_OUTPUT.PUT_LINE('Employ introuvable : ' ||
13                               p_nom);
14     ELSE
15         DBMS_OUTPUT.PUT_LINE(

```

```
15      'Salaire de ' || p_nom || ' augment de ' || p_taux
16      || '%.'
17    );
18    COMMIT;
19  END IF;
20 END;
21 /
22 BEGIN
23   augmenter_salaire('Ali', 10);
24 END;
25 /
```

Listing 6.7 – Procédure : augmentation de salaire par nom

Variante avec clé primaire ENUM :

```
1 CREATE OR REPLACE PROCEDURE augmenter_salaire_enum(
2   p_enum IN VARCHAR2,
3   p_taux IN NUMBER
4 ) IS
5 BEGIN
6   UPDATE EMPLOYES
7   SET   SALARY = SALARY * (1 + p_taux / 100)
8   WHERE ENUM = p_enum;
9
10  IF SQL%ROWCOUNT = 0 THEN
11    DBMS_OUTPUT.PUT_LINE('ENUM introuvable : ' || p_enum);
12  ELSE
13    COMMIT;
14    DBMS_OUTPUT.PUT_LINE('Salaire mis      jour pour ' ||
15                          p_enum);
16  END IF;
17 END;
18 /
```

Listing 6.8 – Procédure : augmentation par ENUM (clé primaire)

Questions supplémentaires :

- L'utilisation de la clé primaire **ENUM** garantit l'unicité de la ligne mise à jour. Passer par le nom expose au risque d'affecter plusieurs employés homonymes.
- **SQL%ROWCOUNT** vaut le nombre de lignes réellement affectées par le dernier **UPDATE**.

Exercice 5 : Conversion en Euros

```
1 CREATE OR REPLACE FUNCTION fn_mad_to_eur(  
2   p_montant IN NUMBER  
3 ) RETURN NUMBER IS  
4   c_taux CONSTANT NUMBER := 11;  
5 BEGIN  
6   RETURN ROUND(p_montant / c_taux, 2);  
7 END;  
8 /  
9  
10 SELECT fn_mad_to_eur(5500) AS montant_eur FROM DUAL;
```

Listing 6.9 – Fonction : conversion dirhams vers euros

Variante avec taux variable :

```
1 CREATE OR REPLACE FUNCTION fn_mad_to_eur(  
2   p_montant IN NUMBER,  
3   p_taux     IN NUMBER DEFAULT 11  
4 ) RETURN NUMBER IS  
5 BEGIN  
6   RETURN ROUND(p_montant / p_taux, 2);  
7 END;  
8 /
```

Listing 6.10 – Fonction surchargée : taux paramétrable

Questions supplémentaires :

- Une fonction peut contenir un COMMIT ou ROLLBACK mais c'est fortement déconseillé : Oracle interdit son appel dans un SELECT si elle contient des DML avec transaction.
- Cette fonction peut être utilisée dans une clause WHERE : WHERE fn_mad_to_eur(SALARY) > 500.

Exercice 6 : Employés au-dessus d'un Seuil de Salaire

```
1 CREATE OR REPLACE PROCEDURE pr_salaires_sup(  
2   p_seuil IN NUMBER,  
3   p_ordre IN VARCHAR2 DEFAULT 'ASC'  
4 ) IS  
5 BEGIN  
6   IF p_seuil < 0 THEN  
7     DBMS_OUTPUT.PUT_LINE('Erreur : le montant ne peut pas  
8       tre n gatif.');
```



```
9  END IF;
10
11  FOR rec IN (
12      SELECT ENAME, SALARY, DEPT FROM EMPLOYES
13      WHERE SALARY > p_seuil
14      ORDER BY CASE WHEN p_ordre = 'DESC' THEN SALARY END
15                  DESC,
16                  CASE WHEN p_ordre = 'ASC' THEN SALARY END ASC
17  ) LOOP
18      DBMS_OUTPUT.PUT_LINE(
19          rec.ENAME || ' | Salaire : ' || rec.SALARY ||
20          ' | Dept : ' || rec.DEPT
21      );
22  END LOOP;
23 END;
24 /
25 BEGIN
26     pr_salaires_sup(7000, 'DESC');
27 END;
28 /
```

Listing 6.11 – Procédure : affichage des salaires supérieurs à un seuil

Questions supplémentaires :

- Sans curseur explicite, on peut utiliser directement la boucle `FOR rec IN (SELECT ...) LOOP`.
- Si le montant est négatif, la procédure affiche un message d'erreur et s'arrête grâce à l'instruction `RETURN`.

Exercice 7 : Salaire Moyen par Département

```
1  CREATE OR REPLACE FUNCTION fn_sal_moyen_dept(
2      p_dept IN VARCHAR2
3  ) RETURN NUMBER IS
4      v_moyenne NUMBER;
5  BEGIN
6      SELECT AVG(SALARY) INTO v_moyenne
7      FROM EMPLOYES
8      WHERE DEPT = p_dept;
9
10     RETURN NVL(v_moyenne, 0);
11
12 EXCEPTION
13     WHEN NO_DATA_FOUND THEN
```

```

14     RETURN NULL;
15 END;
16 /
17
18 SELECT fn_sal_moyen_dept('D1') AS sal_moyen FROM DUAL;

```

Listing 6.12 – Fonction : salaire moyen d'un département

Questions supplémentaires :

- NVL est nécessaire car AVG retourne NULL sur un ensemble vide; sans lui, la fonction renverrait NULL silencieusement.
- Cette fonction peut tout à fait être appelée dans une procédure qui met à jour une table de statistiques : UPDATE stats SET moy = fn_sal_moyen_dept('D1') WHERE dept = 'D1'.

Exercice 8 : Mise à Jour d'Adresse par Nom

```

1 CREATE OR REPLACE PROCEDURE pr_maj_adresse(
2     p_nom          IN VARCHAR2,
3     p_adresse      IN VARCHAR2
4 ) IS
5 BEGIN
6     IF TRIM(p_adresse) IS NULL THEN
7         DBMS_OUTPUT.PUT_LINE('Erreur : l adresse ne peut pas
8             tre vide. ');
9         RETURN;
10    END IF;
11
12    UPDATE EMPLOYES
13    SET     ADDRESS = p_adresse
14    WHERE  ENAME = p_nom
15    AND    ROWNUM = 1;
16
17    IF SQL%ROWCOUNT = 0 THEN
18        DBMS_OUTPUT.PUT_LINE('Employ introuvable : ' ||
19            p_nom);
20    ELSE
21        COMMIT;
22        DBMS_OUTPUT.PUT_LINE('Adresse mise jour pour ' ||
23            p_nom);
24    END IF;
25 END;
26 /
27
28 CREATE OR REPLACE PROCEDURE pr_maj_adresse_enum(

```

```

26  p_enum      IN VARCHAR2 ,
27  p_adresse   IN VARCHAR2
28 ) IS
29 BEGIN
30  IF TRIM(p_adresse) IS NULL THEN
31      DBMS_OUTPUT.PUT_LINE('Erreur : adresse vide. ');
32      RETURN;
33  END IF;
34
35  UPDATE EMPLOYES SET ADDRESS = p_adresse WHERE ENUM =
      p_enum;
36
37  IF SQL%ROWCOUNT = 0 THEN
38      DBMS_OUTPUT.PUT_LINE('ENUM introuvable : ' || p_enum);
39  ELSE
40      COMMIT;
41  END IF;
42 END;
43 /

```

Listing 6.13 – Procédure : mise à jour d'adresse par nom

Exercice 9 : Nombre d'Employés par Département

```

1  CREATE OR REPLACE FUNCTION fn_nb_employes(
2  p_dept IN VARCHAR2
3  ) RETURN NUMBER IS
4  v_count NUMBER;
5  BEGIN
6  SELECT COUNT(*) INTO v_count
7  FROM EMPLOYES
8  WHERE DEPT = p_dept;
9
10 RETURN v_count;
11 END;
12 /
13
14 SELECT fn_nb_employes('D1') AS nb_emp FROM DUAL;

```

Listing 6.14 – Fonction : nombre d'employés d'un département

Procédure affichant le récapitulatif par département :

```

1  CREATE OR REPLACE PROCEDURE pr_recap_dept IS
2  BEGIN
3      FOR rec IN (
4          SELECT DEPT,

```

```

5          COUNT(*)      AS nb_emp,
6          AVG(SALARY) AS sal_moyen
7      FROM    EMPLOYEES
8      GROUP BY DEPT
9      ORDER BY DEPT
10 ) LOOP
11     DBMS_OUTPUT.PUT_LINE(
12         'Dept : ' || rec.DEPT ||
13         ' | Effectif : ' || rec.nb_emp ||
14         ' | Salaire moyen : ' || ROUND(rec.sal_moyen, 2)
15     );
16 END LOOP;
17 END;
18 /
19
20 BEGIN pr_recap_dept; END;
21 /

```

Listing 6.15 – Procédure : récapitulatif par département

Questions supplémentaires :

- COUNT(*) compte toutes les lignes y compris celles avec des NULL. COUNT(ENUM) exclut les NULL sur ENUM, mais comme c'est une clé primaire, le résultat est identique.

Exercice 10 : Suppression des Petits Salaires

```

1 CREATE OR REPLACE PROCEDURE pr_supprimer_petits_salaires(
2     p_seuil IN NUMBER
3 ) IS
4     v_noms VARCHAR2(500) := '';
5 BEGIN
6     SAVEPOINT avant_suppression;
7
8     FOR rec IN (
9         SELECT ENAME FROM EMPLOYEES WHERE SALARY < p_seuil
10     ) LOOP
11         v_noms := v_noms || rec.ENAME || ' ';
12     END LOOP;
13
14     DELETE FROM EMPLOYEES WHERE SALARY < p_seuil;
15
16     IF SQL%ROWCOUNT = 0 THEN
17         DBMS_OUTPUT.PUT_LINE('Aucun employé sous le seuil de
18             ' || p_seuil);
19         ROLLBACK TO avant_suppression;

```

```

19  ELSE
20      DBMS_OUTPUT.PUT_LINE(
21          SQL%ROWCOUNT || ' employ (s) supprim (s) : ' ||
                v_noms
22      );
23      COMMIT;
24  END IF;
25
26  EXCEPTION
27      WHEN OTHERS THEN
28          ROLLBACK TO avant_suppression;
29          DBMS_OUTPUT.PUT_LINE('Erreur : ' || SQLERRM);
30  END;
31  /
32
33  BEGIN
34      pr_supprimer_petits_salaires(5000);
35  END;
36  /

```

Listing 6.16 – Procédure : suppression des salaires inférieurs à un seuil

Questions supplémentaires :

- Un **COMMIT** dans une procédure appelée par un programme externe valide définitivement la transaction, empêchant l'appelant de faire un **ROLLBACK** global en cas d'erreur ultérieure.
- Le **SAVEPOINT** permet un retour partiel en cas d'erreur sans annuler toute la session.

Synthèse : Procédure vs Fonction

Critère	Procédure	Fonction
Retourne une valeur	Non (sauf OUT)	Oui (RETURN)
Appel dans une requête SQL	Non	Oui
Transaction possible	Oui (déconseillé)	Déconseillé
Utilisation typique	Actions, traitements	Calculs, transformations

Conclusion

Ce TP a permis de maîtriser les sous-programmes PL/SQL dans leur ensemble :

- Les procédures sont adaptées aux traitements avec effets de bord (modifications de tables, affichages) et peuvent communiquer des résultats via des paramètres **OUT** ou **IN OUT**.
- Les fonctions sont conçues pour retourner une valeur et s'intégrer directement dans des expressions SQL ou des clauses **WHERE**.

- La gestion des exceptions (`NO_DATA_FOUND`, `OTHERS`) et l'utilisation de `SAVEPOINT` garantissent la robustesse des sous-programmes face aux erreurs.
- Le choix entre procédure et fonction doit être guidé par le besoin : action vs calcul, appel SQL vs appel procédural.

Chapitre 7

TP 7 — Les Triggers PL/SQL sous Oracle

Ce TP a pour objectif de maîtriser les triggers PL/SQL Oracle, notamment :

- Comprendre les types de triggers : BEFORE, AFTER, INSTEAD OF, ligne vs instruction
- Utiliser les pseudo-enregistrements :NEW et :OLD
- Implémenter des contraintes d'intégrité complexes via triggers
- Gérer les exceptions personnalisées avec RAISE_APPLICATION_ERROR
- Détecter des circuits dans un graphe de prérequis

Code Complet de Création des Tables

```
1 DROP TABLE passer      CASCADE CONSTRAINTS;
2 DROP TABLE examen      CASCADE CONSTRAINTS;
3 DROP TABLE inscription  CASCADE CONSTRAINTS;
4 DROP TABLE prerequis    CASCADE CONSTRAINTS;
5 DROP TABLE module       CASCADE CONSTRAINTS;
6 DROP TABLE etudiant     CASCADE CONSTRAINTS;
7
8 CREATE TABLE etudiant (
9     id_etudiant          NUMBER PRIMARY KEY,
10    nom                   VARCHAR2(50),
11    prenom                VARCHAR2(50),
12    date_inscription      DATE,
13    moyenne               NUMBER
14 );
15
16 CREATE TABLE module (
```

```
17  code_module  VARCHAR2(10) PRIMARY KEY,
18  intitule     VARCHAR2(100),
19  effecMax     NUMBER,
20  note_min     NUMBER
21 );
22
23 CREATE TABLE prerequis (
24  module       VARCHAR2(10) REFERENCES module(code_module),
25  prerequis    VARCHAR2(10) REFERENCES module(code_module),
26  note_min     NUMBER,
27  PRIMARY KEY (module, prerequis)
28 );
29
30 CREATE TABLE inscription (
31  id_etudiant  NUMBER REFERENCES
32  etudiant(id_etudiant),
33  code_module  VARCHAR2(10) REFERENCES module(code_module),
34  date_inscription DATE,
35  PRIMARY KEY (id_etudiant, code_module)
36 );
37
38 CREATE TABLE examen (
39  id_examen    NUMBER PRIMARY KEY,
40  code_module  VARCHAR2(10) REFERENCES module(code_module),
41  date_examen  DATE,
42  coefficient  NUMBER
43 );
44
45 CREATE TABLE passer (
46  id_etudiant  NUMBER REFERENCES etudiant(id_etudiant),
47  id_examen    NUMBER REFERENCES examen(id_examen),
48  note         NUMBER,
49  PRIMARY KEY (id_etudiant, id_examen)
50 );
51
52 CREATE TABLE log_erreurs (
53  id_log       NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
54  message      VARCHAR2(500),
55  date_log     DATE DEFAULT SYSDATE
56 );
```

Listing 7.1 – Suppression et recréation complète des tables

Exercices Supplémentaires Proposés

Contrainte 10 : Interdiction de re-s'inscrire à un module déjà validé

Un étudiant ne peut s'inscrire à un module s'il l'a déjà validé (note obtenue $\geq$ note_min du module).

```

1 CREATE OR REPLACE TRIGGER
     trg_inscription_module_deja_valide
2 BEFORE INSERT ON inscription
3 FOR EACH ROW
4 DECLARE
5     v_note_max NUMBER;
6     v_note_min NUMBER;
7 BEGIN
8     SELECT note_min INTO v_note_min
9     FROM   module
10    WHERE  code_module = :NEW.code_module;
11
12     SELECT MAX(pa.note) INTO v_note_max
13     FROM   passer pa
14    JOIN    examen e ON pa.id_examen = e.id_examen
15    WHERE  pa.id_etudiant = :NEW.id_etudiant
16    AND    e.code_module = :NEW.code_module;
17
18     IF v_note_max IS NOT NULL AND v_note_max >= v_note_min
19     THEN
20         RAISE_APPLICATION_ERROR(-20010,
21             'L etudiant ' || :NEW.id_etudiant ||
22             ' a d j   valid le module ' || :NEW.code_module ||
23             ' avec une note de ' || v_note_max);
24     END IF;
25
26 EXCEPTION
27     WHEN NO_DATA_FOUND THEN NULL;
28 END;
/
```

Listing 7.2 – Trigger : interdiction de re-inscription à un module validé

Contrainte 11 : Pas de deux examens dans un intervalle de 7 jours

Lors de l'insertion d'un examen, vérifier qu'il n'existe pas déjà un examen pour le même module dans les 7 jours précédents ou suivants.

```

1 CREATE OR REPLACE TRIGGER trg_examen_check_conflit_date
2 BEFORE INSERT ON examen
3 FOR EACH ROW
```

```

4 DECLARE
5     v_conflit NUMBER;
6 BEGIN
7     SELECT COUNT(*) INTO v_conflit
8     FROM   examen
9     WHERE  code_module = :NEW.code_module
10    AND    ABS(:NEW.date_examen - date_examen) <= 7;
11
12    IF v_conflit > 0 THEN
13        RAISE_APPLICATION_ERROR(-20011,
14            'Conflit de planning : un examen pour le module ' ||
15            :NEW.code_module ||
16            ' existe d j    dans un intervalle de 7 jours. ');
17    END IF;
18 END;
19 /

```

Listing 7.3 – Trigger : contrôle de conflit de planning examen

Contrainte 12 : Moyenne calculée uniquement si tous les examens sont passés

La moyenne d'un étudiant ne doit être calculée que s'il a passé tous les examens des modules auxquels il est inscrit. Sinon, la moyenne reste NULL et un avertissement est journalisé dans log_erreurs.

```

1 CREATE OR REPLACE TRIGGER trg_passer_moyenne_conditionnelle
2 AFTER INSERT OR UPDATE OR DELETE ON passer
3 FOR EACH ROW
4 DECLARE
5     PRAGMA AUTONOMOUS_TRANSACTION;
6     v_id_etudiant NUMBER;
7     v_nb_inscrits NUMBER;
8     v_nb_passes   NUMBER;
9     v_moyenne     NUMBER;
10 BEGIN
11     IF INSERTING OR UPDATING THEN
12         v_id_etudiant := :NEW.id_etudiant;
13     ELSE
14         v_id_etudiant := :OLD.id_etudiant;
15     END IF;
16
17     SELECT COUNT(*) INTO v_nb_inscrits
18     FROM   inscription
19     WHERE  id_etudiant = v_id_etudiant;
20

```

```

21  SELECT COUNT(DISTINCT e.code_module) INTO v_nb_passes
22  FROM    passer pa
23  JOIN    examen e ON pa.id_examen = e.id_examen
24  WHERE   pa.id_etudiant = v_id_etudiant
25  AND     pa.note IS NOT NULL;
26
27  IF v_nb_passes < v_nb_inscrits THEN
28      INSERT INTO log_erreurs(message)
29      VALUES ('Etudiant ' || v_id_etudiant ||
30              ' : moyenne non calcul e , examens manquants
31              (' ||
32              v_nb_passes || '/' || v_nb_inscrits || ').');
33      COMMIT;
34      UPDATE etudiant SET moyenne = NULL
35      WHERE   id_etudiant = v_id_etudiant;
36  ELSE
37      SELECT SUM(pa.note * e.coefficient) /
38             SUM(e.coefficient)
39      INTO    v_moyenne
40      FROM    passer pa
41      JOIN    examen e ON pa.id_examen = e.id_examen
42      WHERE   pa.id_etudiant = v_id_etudiant
43      AND     pa.note IS NOT NULL;
44
45      UPDATE etudiant SET moyenne = v_moyenne
46      WHERE   id_etudiant = v_id_etudiant;
47  END IF;
48  END;
49  /

```

Listing 7.4 – Trigger : moyenne conditionnelle avec journalisation

Contrainte 13 : Interdiction de supprimer un module utilisé comme prérequis

Un module ne peut être supprimé s'il est référencé comme prérequis par un autre module.

```

1  CREATE OR REPLACE TRIGGER trg_module_no_delete_si_prerequis
2  BEFORE DELETE ON module
3  FOR EACH ROW
4  DECLARE
5      v_count NUMBER;
6  BEGIN
7      SELECT COUNT(*) INTO v_count
8      FROM    prerequis

```

```
9  WHERE prerequis = :OLD.code_module;  
10  
11  IF v_count > 0 THEN  
12      RAISE_APPLICATION_ERROR(-20013,  
13          'Impossible de supprimer le module ' ||  
14          :OLD.code_module ||  
15          ' : il est r f renc comme pr requis par ' ||  
16          v_count || ' module(s).');  
17  END IF;  
18  END;  
19  /
```

Listing 7.5 – Trigger : protection de suppression d'un module prérequis

Ce TP a démontré la puissance des triggers PL/SQL pour implémenter des règles métier complexes au niveau de la base de données :

- Les triggers **BEFORE** permettent de bloquer une opération invalide avant qu'elle ne soit appliquée, via **RAISE_APPLICATION_ERROR**.
- Les triggers **AFTER** sont adaptés aux mises à jour en cascade (comme le recalcul de la moyenne) sans interférer avec l'opération principale.
- **PRAGMA AUTONOMOUS_TRANSACTION** permet de journaliser des événements indépendamment de la transaction principale.
- La détection de circuits dans un graphe de prérequis illustre l'utilisation de fonctions récursives appelées depuis un trigger.
- Les triggers doivent être utilisés avec parcimonie : un abus peut dégrader les performances et rendre la logique métier difficile à maintenir.